

Relatório – Disciplina SD212
Atividade 25-05-2026
Cache Mapeada Diretamente Simples

Este relatório documenta a implementação e simulação de uma cache mapeada diretamente simples, desenvolvida como atividade prática da disciplina de Organização e Arquitetura de Computadores (SD212). O objetivo principal é fixar os conceitos fundamentais de memória cache: divisão do endereço em tag e índice, funcionamento do bit de validade, detecção de hit e miss, e substituição de blocos por conflito.

A cache mapeada diretamente é a organização mais simples de cache: cada bloco da memória principal possui exatamente uma posição possível na cache, determinada pelos bits de índice do endereço. Essa característica facilita a implementação em hardware, mas pode gerar conflitos quando dois endereços distintos competem pela mesma posição.

- **Especificação Do Sistema**

A Tabela 1 apresenta os parâmetros que definem a organização da cache implementada neste exercício.

Tabela 1 — Parâmetros da cache

Parâmetro	Valor
Tipo de cache	Mapeada diretamente
Número de blocos	4
Tamanho do bloco	1 palavra
Largura do endereço	4 bits
Largura do dado	8 bits
Bits de índice	2 bits (bits [1:0])
Bits de tag	2 bits (bits [3:2])

- **Divisão de Endereço:**

Endereço[3:0] = TAG [3:2] | ÍNDICE [1:0]

Exemplo prático: o endereço 1011 é decomposto em tag = 10 (bits mais significativos, posição [3:2]) e índice = 11 (bits menos significativos, posição [1:0]), indicando que o bloco deve ser armazenado na posição 3 da cache e identificado pela tag 10.

- **Implementação em verilog**

O módulo cache_simples foi implementado com a seguinte interface:

Entradas:

1. clk — sinal de clock do circuito
2. reset — reinicializa todos os bits de validade
3. read — habilita operação de leitura
4. address[3:0] — endereço de memória de 4 bits
5. mem_data[7:0] — dado vindo da memória principal em caso de miss

Saídas:

6. cache_data[7:0] — dado retornado pela cache
7. hit — sinaliza acerto na cache
8. miss — sinaliza falha na cache

- **Estruturas internas**

Internamente, o módulo mantém três vetores com quatro posições cada, indexados pelo campo índice do endereço:

9. valid[0:3] — vetor de bits de validade; inicializado em 0 após reset
10. tag_array[0:3] — vetor de tags (2 bits cada) armazenadas na cache
11. data_array[0:3] — vetor de dados (8 bits cada) correspondentes às tags

- **Logica de Funcionamento**

O módulo opera na borda de subida do clock. Quando reset é acionado, todos os bits de validade são zerados. Quando read é habilitado, o endereço é dividido em tag e index. A lógica verifica duas condições para declarar hit: o bit de validade da posição deve ser 1 e a tag armazenada deve ser igual à tag do endereço. Caso uma das condições falhe, ocorre miss e o bloco é carregado da memória principal, atualizando valid, tag_array e data_array na posição correspondente

- **Sequência de Teste e Resultados**

Tabela 2 — Sequência de teste

Acesso	Endereço	Dado (memória)
1	1011	8'hA1

2	1011	8'hA1
3	0011	8'hB2
4	0011	8'hB2
5	1011	8'hA1

- **Log Da Simulação**

```
Acesso 1 | Endereço: 1011 | Tag: 10 | Índice: 11 | Resultado: MISS | Dado: a1
Acesso 2 | Endereço: 1011 | Tag: 10 | Índice: 11 | Resultado: HIT | Dado: a1
Acesso 3 | Endereço: 0011 | Tag: 00 | Índice: 11 | Resultado: MISS | Dado: b2
Acesso 4 | Endereço: 0011 | Tag: 00 | Índice: 11 | Resultado: HIT | Dado: b2
Acesso 5 | Endereço: 1011 | Tag: 10 | Índice: 11 | Resultado: MISS | Dado: a1
Total: 5 acessos | Hits: 2 | Misses: 3 | Taxa de acerto: 40%
```

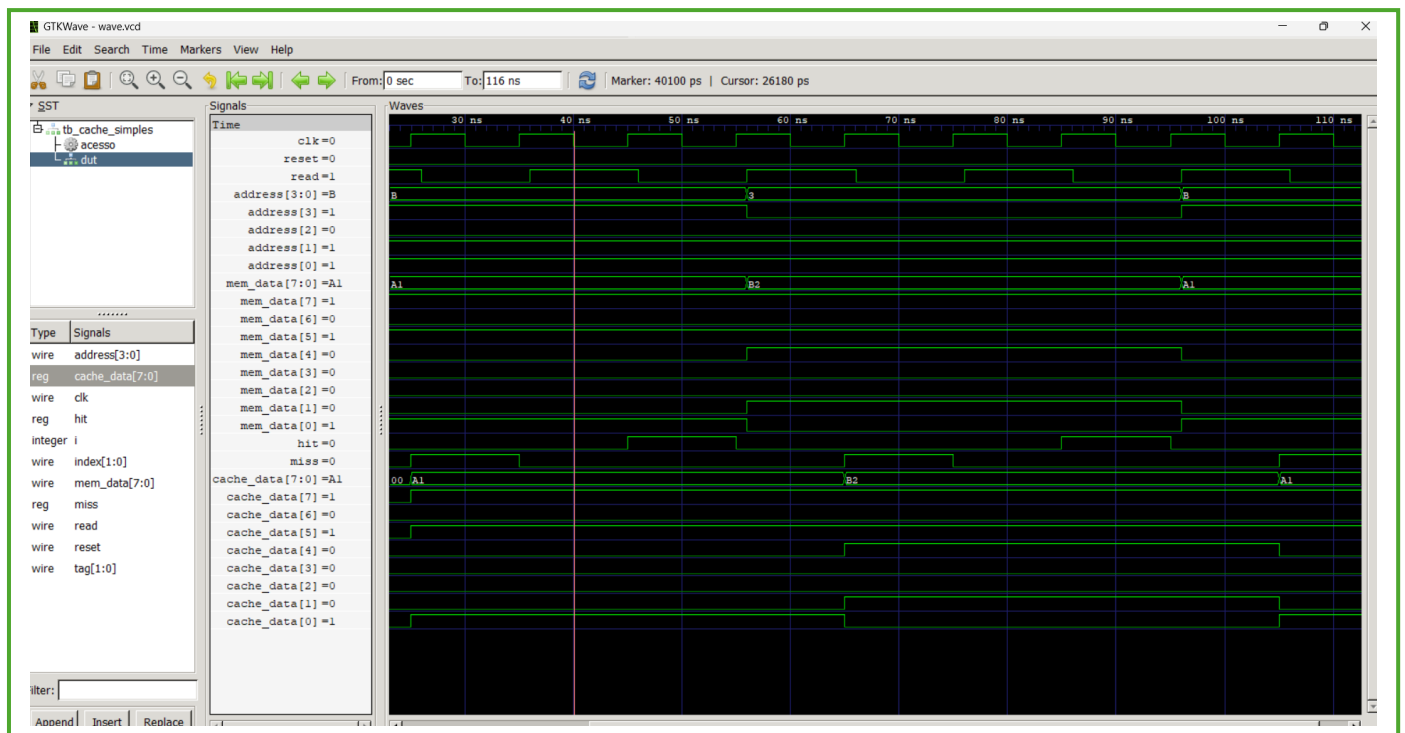
- **Resultados Esperados vs obtido**

Tabela 3 — Resultados detalhados

Ac.	Endereço	Tag	Índice	Resultado	Justificativa
1	1011	10	11	MISS	Primeiro acesso — cache vazia (valid=0)
2	1011	10	11	HIT	Tag 10 já armazenada na posição 11
3	0011	00	11	MISS	Mesma posição, tag diferente (conflito)
4	0011	00	11	HIT	Tag 00 agora armazenada na posição 11
5	1011	10	11	MISS	Tag 10 foi substituída pela 00 no acesso 3

- **Captura de Tela – Waves**

A Figura 1 apresenta a captura de tela da simulação no GTKWave, confirmando visualmente os resultados obtidos. É possível observar os sinais de clk, read, address, mem_data, hit, miss e cache_data ao longo do tempo, com os pulsos de hit ocorrendo nos acessos 2 e 4, e os pulsos de miss nos acessos 1, 3 e 5, exatamente conforme esperado pela especificação.



1. Qual parte do endereço representa a tag?

Os 2 bits mais significativos do endereço (bits [3:2]) representam a tag. Com um endereço de 4 bits e 4 blocos na cache, são necessários $\log_2(4) = 2$ bits para indexar as posições, restando os outros 2 bits para a tag. Exemplo: endereço 1011 $\rightarrow$ tag = 10.

2. Qual parte do endereço representa o índice?

Os 2 bits menos significativos do endereço (bits [1:0]) representam o índice. Esses bits determinam diretamente em qual das 4 posições da cache o bloco será armazenado. Exemplo: endereço 1011 $\rightarrow$ índice = 11, correspondendo à posição 3 da cache.

3. Por que o primeiro acesso sempre gera miss?

Ao inicializar o sistema, o sinal de reset zera todos os bits de validade ($\text{valid}[i] = 0$ para todo i). A lógica de verificação de hit exige que $\text{valid}[\text{index}] = 1$ como primeira condição. Como todos os bits estão em 0, nenhuma posição é considerada válida, resultando obrigatoriamente em miss no primeiro acesso a qualquer endereço. O dado precisa ser carregado da memória principal e a posição correspondente é preenchida e validada.

4. Por que o endereço 0011 gera miss no acesso 3, mesmo possuindo o mesmo índice do endereço 1011?

Na cache mapeada diretamente, cada posição (determinada pelo índice) armazena apenas um bloco por vez, identificado pela sua tag. Os endereços 1011 (tag=10, índice=11) e 0011 (tag=00, índice=11) mapeiam para a mesma posição 11, mas possuem tags distintas. No acesso 3, a posição 11 continha a tag 10 (do acesso 1). Ao comparar com a tag 00 do endereço 0011, a

comparação falha e ocorre miss. O fenômeno é denominado conflito de mapeamento: dois endereços distintos competem pela mesma posição, causando substituições mútuas. Isso explica também o miss do acesso 5: a tag 10 foi sobrescrita pela tag 00 no acesso 3.

5. O que aconteceria se a cache tivesse 8 blocos em vez de 4?

Com 8 blocos, seriam necessários 3 bits de índice ($\log_2(8) = 3$). Como o endereço ainda possui apenas 4 bits, restaria somente 1 bit para a tag. A nova divisão seria: TAG[3] | ÍNDICE[2:0]. Com essa configuração, os endereços 1011 (índice=011, tag=1) e 0011 (índice=011, tag=0) continuariam mapeando para o mesmo índice 011, logo o conflito persistiria nesta sequência específica. Entretanto, em workloads mais amplos, o aumento no número de blocos reduz significativamente a probabilidade de conflito entre endereços distintos, aumentando a taxa de acerto geral da cache.